

Analisi delle Prestazioni di Inserimento in Alberi Binari di Ricerca

Londero Stefano
172585

Grasso Federico
173731

II semestre 2026

1 Introduzione

L'obiettivo del progetto è confrontare tra Alberi Binari di Ricerca (BST), Alberi (AVL) e Red-Black (RB) il tempo medio di inserimento di una chiave quando l'albero contiene n chiavi e, ottenere per diversi valori di n , stime robuste dei tempi di esecuzione.

2 Misurazione

Per la misurazione abbiamo utilizzato metodi come, per esempio, `perf_counter()` per misurare il tempo di esecuzione dell'inserimento di chiavi in alberi BST, AVL e RB. Abbiamo eseguito più iterazioni per ogni valore di n per ottenere stime robuste dei tempi di esecuzione. Abbiamo anche utilizzato grafici per visualizzare i risultati e confrontare le prestazioni dei diversi tipi di alberi. I grafici mostrano il tempo medio di inserimento in funzione del numero di chiavi presenti nell'albero, evidenziando le differenze tra BST, AVL e RB.

3 Opzione C

Come opzione abbiamo scelto la C, dove l'idea è quella di costruire, per ciascun valore di n , un albero di dimensione n e applicare su questo una sequenza di operazioni di inserimento e cancellazione che mantengano l'albero bilanciato. In questo modo, possiamo osservare come le prestazioni di inserimento variano in un contesto dinamico, dove l'albero subisce modifiche continue. Questo approccio ci permette di valutare non solo il tempo di inserimento, ma anche l'efficienza complessiva dell'albero nel mantenere la sua struttura bilanciata durante le operazioni di inserimento e cancellazione.

3.1 Algoritmo

Il progetto implementa un sistema di misurazione delle prestazioni per tre diverse strutture dati ad albero: Alberi Binari di Ricerca (BST), Alberi AVL e Alberi Rosso-Nero (RBT). L'obiettivo è quantificare il tempo di inserimento di una chiave in ciascun tipo di albero al variare della dimensione dell'albero stesso.

3.2 Struttura del Progetto

Il progetto è organizzato nei seguenti moduli:

- **ExperimentSetup**: Configura i parametri dell'esperimento, calcolando una progressione geometrica di valori di n da testare (da 1000 a 10.000.000 elementi) e inizializzando gli alberi con n chiavi casuali.
- **ExperimentRunner**: Esegue una serie di esperimenti controllati per ciascun tipo di albero e per ogni valore di n .
- **BST, AVL, RBT**: Implementano le tre strutture dati con operazioni di inserimento, ricerca e rimozione.
- **KeyManager**: Gestisce l'insieme di chiavi utilizzate negli esperimenti.
- **Plotter**: Visualizza i risultati degli esperimenti tramite grafici.

3.3 Algoritmo di Misurazione

Per ogni valore di n nella progressione geometrica, l'algoritmo esegue i seguenti passi:

1. Inizializzazione: Viene creato un albero vuoto (BST, AVL o RBT).
2. Popolazione iniziale: L'albero viene popolato con n chiavi casuali generate da una lista predefinita di chiavi.
3. Ciclo di misurazione: Per 10 iterazioni (configurabili):
 - (a) Viene selezionata una chiave disponibile dall'insieme delle chiavi non ancora inserite.
 - (b) Si misura il tempo di inserimento di questa chiave nell'albero utilizzando `time.perf_counter()`.
 - (c) Viene rimossa casualmente una chiave dall'albero per mantenere la dimensione costante e simulare un carico dinamico.
4. Calcolo della mediana: Viene calcolata la mediana dei 10 tempi di inserimento misurati, che rappresenta una stima robusta delle prestazioni.

3.4 Caratteristiche Implementative

- **Progressione geometrica**: I valori di n sono generati secondo una progressione geometrica per coprire uniformemente lo spazio logaritmico tra il minimo e il massimo.
- **Bilanciamento dinamico**: AVL e RBT mantengono automaticamente il bilanciamento dell'albero dopo ogni inserimento e rimozione.
- **Gestione della memoria**: Dopo ogni esperimento completo per un valore di n , la memoria viene liberata mediante garbage collection per evitare interferenze tra esperimenti successivi.
- **Raccolta dati robusta**: L'uso della mediana invece della media riduce l'impatto di misurazioni anomale e fornisce stime più affidabili.